Topic 4: Spatial Dependency Modelling in Multivariate Time Series

Step 2: Models without spatial information

What this step is about

This step builds on Step 1. Continue from the work you did there: the same dataset and choices, the code you wrote, and the feedback you received at the meeting. In this step you build the first working version of your experiments. The results you get here are the reference point: everything you build later will be compared against them, so it is important that they are correct and that you understand them.

Note: the tasks below are a general guide, not a fixed recipe. The exact details depend on your results from Step 1 and on what we discuss at the meeting, so some tasks may be adjusted, skipped or replaced. If your Step 1 results suggest a different direction, bring it up.

Work through the tasks in order and do as much as you can. Not everyone will finish everything, and that is fine. What matters is that what you did is done carefully and that you can explain it. Bring the tasks you did not get to, or got stuck on, as questions to the meeting.

What you should have at the end of this step

- **A 5-minute presentation** at the next meeting, covering the steps you followed from this document, your main results, and the questions or open issues you want to discuss.
- **Notebook(s)** with your results: tables and plots that are labelled and commented. A plot without a sentence explaining what it shows does not count.
- **Reusable Python code** for training and evaluating, building on the code from Step 1, written as functions or classes in `src/`, not only as notebook cells.
- **[Optional] A Git repository** with a clear structure and a `README.md` explaining how to run your experiments.

Good practices

- **Same rules for every model.** Use the same data split, the same input length, the same forecast horizon and the same error measures for all models. Otherwise the comparison means nothing.
- **Validation for decisions, test only at the end.** Choose settings (learning rate, model size, when to stop training) by looking at the validation error. Use the test data only to report the final numbers.
- **Start small.** First run everything on a small piece of the data and for a short time, to check that the code works. Only then run the full experiments.
- **Keep a simple log of experiments.** A simple document where you write down, for each run, the model, its settings and its results is enough. After a few weeks you will not remember which run was which.
- **Fix the random seed,** so that running the same code again gives the same numbers.

Recommended tools

Python, Jupyter, pandas, NumPy, Matplotlib or Seaborn, PyTorch (for neural networks), statsmodels, scikit-learn. Topic-specific tools are listed in the next section.

Your topic: step 2 tasks

Goal of step 2: Train models that forecast each sensor using only its own history and ignore the road graph. They give the lower bound: the graph-based models in the next step must do better to show that information about neighbours really helps.

Tools

- **BasicTS:** https://github.com/GestaltCogTeam/BasicTS (includes standard baselines and evaluation for traffic datasets).
- **PyTorch tutorials:** https://pytorch.org/tutorials/ (if you write the models yourself).
- **Optuna:** https://optuna.org/ (optional, for hyperparameter tuning).

Your choices

- **Dataset:** the same one you used in Step 1.
- **Models:** choose at least three, including at least one simple one, from: linear, MLP, LSTM, GRU, TCN (a convolutional network built for sequences) and LightGBM.
- **Input length and features:** how many past values you use, and which calendar inputs you test.
- **Analysis:** which sensors, roads or times of day you look at more closely.

Background

- **What "without spatial information" means:** to forecast a sensor, the model sees only that sensor's past. One model is shared by all sensors (applied to each of them separately), so it learns general traffic behaviour, but nothing about the neighbours.
- **Some of the models, in simple words:** a **linear** model (weighted sum of the last 12 values), an **MLP** (a simple neural network) and an **LSTM** (a recurrent network that reads the series step by step and keeps a memory).
- **How traffic papers report results:** MAE, RMSE and MAPE (the average error in percent) at 3, 6 and 12 steps ahead (15, 30 and 60 minutes), skipping the missing values.
- **Published numbers:** the DCRNN paper you read in Step 1 has a results table for METR-LA and PEMS-BAY that includes simple baselines and an LSTM.

Tasks

1. **Prepare the data for the models.** Start from the examples built in Step 1 (12 values in, 12 values out, for all sensors).
   - Scale the values using z-score with statistics from the training data only.
   - Reshape so that each sensor becomes a separate example: the model gets 12 values of one sensor and predicts the next 12 values of the same sensor.
2. **Write a training and evaluation loop** (or use the one in BasicTS):

  - compute the validation error after each pass over the data (epoch), stop when it no longer improves (this is called **early stopping**) and keep the best version;
  - compute MAE, RMSE and MAPE in the original units at 3, 6 and 12 steps ahead, skipping missing values.
3. **Train and compare your chosen models,** starting with the simplest one. Keep them small.
4. **Select the input features.** Start with the past 12 values only, then add the time-of-day and day-of-week columns from Step 1 (encoded as sine/cosine) one at a time, and keep what lowers the validation error. Traffic depends strongly on the time of day, so this often helps a lot.
   - Also try a longer input (for example 24 or 36 past values) to see whether more history helps.
5. **Tune the hyperparameters. Hyperparameters** are the settings you choose before training, such as the learning rate, the number of layers, the number of units, the batch size or the input length. For each model, try a few values for the most important ones and keep the combination with the lowest validation error. A small grid search is enough, or you can use a tool such as Optuna.
   - Give every model the same tuning budget (for example, the same number of tries), so the comparison stays fair, and write down the values you chose.
6. **Compare** with the simple rules from Step 1 and with the published numbers. If you are close to the published results, your setup is fine.
7. **Find where the models fail.** Compute the error for each sensor and draw the graph with NetworkX, colouring each sensor by its error. Are the hardest sensors in particular places?
8. **Error by time of day.** Compare rush hours with the night. Errors are usually larger when traffic changes quickly, and these are exactly the moments when knowing what the neighbours see should help the most.
9. **Repeat with different seeds.** Run the main models 3 times with different random seeds and report the average and the variation. Also note training time and number of parameters.
10. **Optional: all sensors as input.** Train an MLP that takes the last 12 values of all sensors at once and predicts all sensors. It uses the other sensors, but without the graph. Does it help or hurt? This is an interesting comparison for the next step.

Questions your presentation should answer

- How much better are the models than the simple rules from Step 1?
- At which sensors and at what times of day do they fail the most?
- Which input features help, and how much did tuning improve the models?
- If you did the optional task: does giving the model all sensors, without the graph, help?

Not part of this step: graph neural networks (next step), Transformers.